Chapter 7. Training Deep Networks: SGD, Initialisation, Normalisation

Mini-batch SGD, activations, weight initialisation, learning rates, batch normalisation

Contents

1 Full, Stochastic and Mini-Batch Gradients **2**

2 Activation Functions Revisited **2**

3 Weight Initialisation **2**

3.1 Why the scale matters . . . 2

3.2 Zero and symmetric initialisation . . . 3

3.3 Xavier and He initialisation . . . 3

3.4 Variance of a sigmoid output (2025 midterm Q2.2) . . . 4

4 Learning-Rate Schedules **4**

5 Batch Normalisation **5**

5.1 Training-time computation . . . 5

5.2 Test time . . . 5

6 Dropout **6**

1 Full, Stochastic and Mini-Batch Gradients

For $f(\boldsymbol{\theta}) = \frac{1}{N}\sum_{i=1}^{N} f_i(\boldsymbol{\theta})$, the full gradient costs N per-sample gradients. Choosing I uniformly, $\mathbf{g} = \nabla f_I$ is **unbiased**: $\mathbb{E}[\mathbf{g} \mid \boldsymbol{\theta}] = \frac{1}{N}\sum_i \nabla f_i = \nabla f$. A mini-batch averages b such gradients.

Theorem 7.1: mini-batch variance

If the b indices are drawn independently with replacement and a single draw has conditional covariance Σ, the mini-batch gradient has covariance Σ/b. Without replacement it is $\frac{N-b}{b(N-1)}\Sigma$ (with Σ normalised by N).

Proof. With replacement the b terms are iid with covariance Σ; the covariance of their average is $\frac{1}{b^2}\cdot b\Sigma$. Without replacement the pairwise covariances are $-\Sigma/(N-1)$, which gives the finite-population correction. ■

Example 7.2: unbiased but noisy

$f_i(w) = \frac{1}{2}(w - y_i)^2$ with $\mathbf{y} = (1, 3, 5)$ at $w = 0$: per-sample gradients $-1, -3, -5$, full gradient -3. A single random gradient averages to -3; a two-sample batch without replacement gives $-2, -3, -4$, again averaging to -3.

One *iteration* is one parameter update; one *epoch* is one pass through the data (1000 samples, batch 100: 10 iterations per epoch). Full batch: deterministic but expensive; single sample: cheap but noisy; mini-batch: a compromise.

2 Activation Functions Revisited

Sigmoid. Saturates: $\sigma' \le 1/4$ and $\sigma' \to 0$ for large $|z|$. Outputs are positive, so for the next layer $h = W\mathbf{z}$ (with $\mathbf{z}$ the positive sigmoid outputs) $\partial L/\partial W_{ij} = \delta_i z_j$ has the sign of δ_i for every j: all weights into a neuron move in the same direction in one step, producing zig-zag paths. **Tanh.** Zero-centred and odd, $\tanh' \in (0, 1]$, but still saturates near ± 1. **ReLU.** Cheap, derivative 1 on the positive side (no saturation there), but units with negative input get zero gradient (dead ReLU) and outputs are not centred. Leaky ReLU, ELU and maxout keep a nonzero slope or add flexibility.

3 Weight Initialisation

3.1 Why the scale matters

`W = s * np.random.randn(Din, Dout)` draws iid $N(0, s^2)$ weights. For $z_j = \sum_{i=1}^{D_{\text{in}}} W_{ji}x_i$ with zero-mean independent weights of variance σ_W^2, independent of inputs with $\mathbb{E}x_i^2 = q$,

$$\mathbb{E}z_j = 0, \qquad \operatorname{Var} z_j = D_{\text{in}}\sigma_W^2 q$$

(cross terms vanish because $\mathbb{E}[W_{ji}W_{jk}] = 0$ for $i \neq k$). For six tanh layers of width 4096:

- $s = 0.01$: $D_{\text{in}}s^2 = 0.41$; activations shrink towards 0 layer by layer, and weight gradients $\propto$ inputs become tiny.
- $s = 0.05$: $D_{\text{in}}s^2 = 10.24$; pre-activations have standard deviation ≈ 3, tanh saturates (histograms pile up at ± 1), $\tanh' \approx 0$, and the backpropagated signal $\partial L/\partial \mathbf{z} = \sigma'(\cdot)W^\top \partial L/\partial \mathbf{h}$ is almost zero: no learning, although the loss is still large. A zero gradient caused by saturation is not a sign of optimality.

3.2 Zero and symmetric initialisation

Theorem 7.3: zero initialisation freezes a tanh MLP (2022 midterm Q5, 2025 midterm Q3)

For $f(\mathbf{x}) = A_L y_{L-1} + b_L$, $y_\ell = \tanh(A_\ell y_{\ell-1} + b_\ell)$, all parameters initialised to 0 and GD on $\frac{1}{2}\sum_i (f(X_i) - Y_i)^2$: $A_1, \dots, A_L, b_1, \dots, b_{L-1}$ stay 0 forever, and $b_L \to \bar{Y}$ if $0 < \eta < 2/N$.

Proof. Induction on the claim "all parameters except b_L are zero". Forward: $\tanh(0) = 0$ so every hidden output is 0 and $f = b_L$. Backward: $\partial/\partial A_L = \sum_i \delta_L^{(i)} y_{L-1}^\top = 0$; $\boldsymbol{\delta}_{L-1} = (A_L^\top \delta_L) \odot \tanh'(0) = 0$ since $A_L = 0$, and inductively all $\boldsymbol{\delta}_\ell = 0$, so $\partial A_\ell = \boldsymbol{\delta}_\ell y_{\ell-1}^\top = 0$, $\partial b_\ell = 0$. Hence GD does not move them. For b_L: $J = \frac{1}{2}\sum (b_L - Y_i)^2$, $b^{(t+1)} - \bar{Y} = (1 - \eta N)(b^{(t)} - \bar{Y})$. ■

Theorem 7.4: duplicate neurons stay duplicates (2021 midterm Q2)

For $f(x) = \sum_{j=1}^p u_j \sigma(a_j x + b_j)$ trained by Adam (or GD, momentum, RMSProp), if $(a_{p-1}, b_{p-1}, u_{p-1}) = (a_p, b_p, u_p)$ at initialisation, they remain equal for all iterations.

Proof. $\partial f/\partial u_j = \sigma(a_j x + b_j)$, $\partial f/\partial a_j = u_j \sigma'(a_j x + b_j) x$, $\partial f/\partial b_j = u_j \sigma'(a_j x + b_j)$; the loss gradient multiplies these by $\partial \ell/\partial f$, which is shared by all neurons. So equal parameters give equal gradient coordinates. Induct on the statement "the paired coordinates of $\boldsymbol{\theta}^k$, m_1^k, m_2^k agree": it holds at $k = 0$ ($m_1^0 = m_2^0 = 0$), and every Adam operation (EMAs, bias corrections, elementwise division) acts coordinatewise. The optimiser state must be part of the induction hypothesis. ■

Hence identical neurons compute the same function forever: random initialisation is needed to break symmetry.

3.3 Xavier and He initialisation

For an approximately linear activation, preserving the second moment requires $D_{\text{in}} \sigma_W^2 = 1$, i.e. $\sigma_W = 1/\sqrt{D_{\text{in}}}$ (often called "Xavier"; Glorot's compromise $2/(D_{\text{in}} + D_{\text{out}})$ also balances the backward pass).

Theorem 7.5: He initialisation (2022 midterm Q6)

Let $y^+ = A\mathbf{x}$, $\mathbf{x} = \text{ReLU}(\mathbf{y})$, $A_{ij} \overset{\text{iid}}{\sim} N(0, \sigma_A^2)$, where $\mathbf{y}$ is independent of A, its coordinates have mean 0, variance 1, are uncorrelated, and $\mathbf{y} \overset{d}{=} -\mathbf{y}$. Then

(i) $\mathbb{E}x_j^2 = \frac{1}{2}$; (ii) $\mathbb{E}y^+ = 0$; (iii) $\mathbb{E}(y_i^+)^2 = \frac{n_{\text{in}}\sigma_A^2}{2}$;

(iv) $y_1^+, \dots, y_{n_{\text{out}}}^+$ are uncorrelated; (v) $y^+ \overset{d}{=} -y^+$.

Thus $\sigma_A^2 = 2/n_{\text{in}}$ preserves the second moment through ReLU layers.

Proof. (i) $\text{ReLU}(y)^2 = y^2 \mathbf{1}\{y > 0\}$; symmetry makes the positive and negative halves of $\mathbb{E}y^2 = 1$ equal ($y = 0$ contributes nothing). (ii) Independence and $\mathbb{E}A_{ij} = 0$. (iii) $\mathbb{E}(y_i^+)^2 = \sum_{j,k} \mathbb{E}[A_{ij}A_{ik}]\mathbb{E}[x_j x_k] = \sum_j \sigma_A^2 \mathbb{E}x_j^2$. (iv) For $i \neq k$ all $\mathbb{E}[A_{ij}A_{kl}] = 0$. (v) $(A, \mathbf{y}) \overset{d}{=} (-A, \mathbf{y})$ since A is independent of $\mathbf{y}$ and symmetric; apply $F(A, \mathbf{y}) = A\,\text{ReLU}(\mathbf{y})$. ■

Remark 7.6: refinements

Leaky ReLU with slope α: $\mathbb{E}\phi(y)^2 = \frac{1+\alpha^2}{2}$ and $\sigma_A^2 = \frac{2}{(1+\alpha^2)n_{\text{in}}}$. ReLU output is not centred: for $z \sim N(0, v)$, $\mathbb{E}\,\text{ReLU}(z) = \sqrt{v/(2\pi)}$, $\mathbb{E}\,\text{ReLU}(z)^2 = v/2$, so He preserves the second moment, not the variance. Backward, $\partial\ell/\partial\mathbf{x} = A^\top \partial\ell/\partial\mathbf{y}^+$ has variance $n_{\text{out}}\sigma_A^2$: forward and backward preservation ask for n_{in} versus n_{out}.

Theorem 7.7: sigmoid layer (2021 midterm Q4)

For $\mathbf{y} = \sigma(A\mathbf{x})$ with $\sigma(t) \approx \frac{1}{2} + \frac{t}{4}$, $\sigma' \approx \frac{1}{4}$, $A_{ij} \overset{\text{iid}}{\sim} N(0, \sigma_A^2)$ independent of $\mathbf{x}$, and x_j uncorrelated with mean $\frac{1}{2}$ and variance 1: $\mathbb{E}y_i = \frac{1}{2}$, $\operatorname{Var} y_i = \frac{5n_{\text{in}}\sigma_A^2}{64}$. If the upstream gradients $\partial\ell/\partial y_i$ have mean 0, variance 1, are uncorrelated and independent of A, then $\partial\ell/\partial x_j \approx \frac{1}{4}\sum_i A_{ij}\partial\ell/\partial y_i$ has mean 0 and variance $\frac{n_{\text{out}}\sigma_A^2}{16}$.

Proof. $\operatorname{Var}(A_{ij}x_j) = \mathbb{E}A_{ij}^2\mathbb{E}x_j^2 = \sigma_A^2(1 + \frac{1}{4})$ and the terms are uncorrelated; multiply by $\frac{1}{16}$. The backward computation is identical with $\mathbb{E}(\partial\ell/\partial y_i)^2 = 1$. ■

Theorem 7.8: convolution layer (2024 midterm Q5)

For a 3×3, padding-1 convolution $Y_{cij} = \sum_{p,q,r} w_{cpqr}\tilde{X}_{p,i+q-2,j+r-2} + b_c$ with zero-mean unit-variance uncorrelated inputs, $w \overset{\text{iid}}{\sim} N(0, \sigma_w^2)$, $b \overset{\text{iid}}{\sim} N(0, \sigma_b^2)$ independent: $\mathbb{E}Y_{cij} = 0$ everywhere, and $\mathbb{E}Y_{cij}^2 = \sigma_w^2 \cdot \#\{\text{in-range inputs}\} + \sigma_b^2$, i.e. $9C_{\text{in}}\sigma_w^2 + \sigma_b^2$ in the interior, $6C_{\text{in}}\sigma_w^2 + \sigma_b^2$ on edges and $4C_{\text{in}}\sigma_w^2 + \sigma_b^2$ at corners.

Proof. Expand the square: weight–weight terms leave $\sigma_w^2\mathbb{E}\tilde{X}_a^2$ on the diagonal, weight–bias terms vanish, and $\mathbb{E}b_c^2 = \sigma_b^2$. Padded entries are zero. ■

3.4 Variance of a sigmoid output (2025 midterm Q2.2)

For smooth f and X with mean μ, Taylor's theorem $f(X) = f(\mu) + f'(\mu)(X-\mu) + \frac{1}{2}f''(\xi)(X-\mu)^2$ gives $\operatorname{Var} f(X) = f'(\mu)^2 \operatorname{Var} X + \text{Error}$, the error collecting terms with f''. For $X \sim N(0,1)$ and $Y = \sigma(wX + b)$, $\operatorname{Var} Y \approx w^2\sigma'(b)^2$: small weights or large $|b|$ (saturation) give small output variance. An exact bound also holds: if f is L-Lipschitz, $\operatorname{Var} f(X) \le \mathbb{E}(f(X) - f(\mu))^2 \le L^2 \operatorname{Var} X$, so $\operatorname{Var} Y \le w^2/16$.

4 Learning-Rate Schedules

The learning rate controls the update size: too large diverges, too small is slow, and a large early rate followed by a smaller late rate refines the solution and reduces gradient noise.

- Step: multiply by 0.1 at fixed epochs (ResNet: 30, 60, 90), giving $\eta_0, 0.1\eta_0, 0.01\eta_0, 0.001\eta_0$.
- Cosine: $\eta_t = \frac{\eta_0}{2}[1 + \cos(\pi t/T)]$; linear: $\eta_t = \eta_0(1 - t/T)$; inverse square root: $\eta_t = \eta_0/\sqrt{t}$.
- Linear warmup: increase from 0 to $\eta_{\max}$ over the first iterations (e.g. 5000), then decay.

5 Batch Normalisation

5.1 Training-time computation

For a mini-batch $X \in \mathbb{R}^{B\times D}$ (rows: samples, columns: features),

$$\mu_j = \frac{1}{B}\sum_i X_{ij}, \quad v_j = \frac{1}{B}\sum_i (X_{ij}-\mu_j)^2, \quad \hat{X}_{ij} = \frac{X_{ij}-\mu_j}{\sqrt{v_j+\epsilon}}, \quad Y_{ij} = \gamma_j \hat{X}_{ij} + \beta_j.$$

γ, β are learned, so the network can choose the output scale and centre (with $\gamma_j = \sqrt{v_j+\epsilon}$, $\beta_j = \mu_j$ it recovers the input for that batch).

Example 7.9: two features

$X = \begin{pmatrix} 1 & 10 \\ 3 & 14 \end{pmatrix}$: $\mu = (2, 12)$, $v = (1, 4)$, $\hat{X} = \begin{pmatrix} -1 & -1 \\ 1 & 1 \end{pmatrix}$ (ϵ ignored). With $\gamma = (2, 3)$, $\beta = (-1, 5)$, $Y = \begin{pmatrix} -3 & 2 \\ 1 & 8 \end{pmatrix}$.

Theorem 7.10: invariances

With $\epsilon = 0$: (i) each column of $\hat{X}$ has mean 0 and mean square 1; (ii) for $z = \mathbf{w}^\top \mathbf{x} + b$ the BN output does not depend on b and is invariant under $\mathbf{w} \mapsto c\mathbf{w}$, $c > 0$; (iii) consequently $\langle \nabla_{\mathbf{w}} L, \mathbf{w} \rangle = 0$.

Proof. (ii) $z - \mu = \mathbf{w}^\top(\mathbf{x} - \bar{\mathbf{x}})$ removes b; scaling $\mathbf{w}$ scales both $z-\mu$ and $\sqrt{v}$ by c. (iii) $h(c) = L(c\mathbf{w})$ is constant, so $0 = h'(1) = \nabla L(\mathbf{w})^\top \mathbf{w}$. ∎

5.2 Test time

At inference the batch may have size one, and the output must not depend on the other samples. During training BN keeps running averages

$$\mu^{\text{run}} \leftarrow m\,\mu^{\text{run}} + (1-m)\mu_{\text{batch}}, \qquad (\sigma^2)^{\text{run}} \leftarrow m\,(\sigma^2)^{\text{run}} + (1-m)\sigma^2_{\text{batch}}$$

(an exponential moving average, as in Adam; PyTorch's `momentum` argument is $1-m$), and at test time uses

$$\hat{x}_{ij} = \frac{x_{ij} - \mu_j^{\text{run}}}{\sqrt{(\sigma_j^2)^{\text{run}} + \epsilon}}, \qquad y_{ij} = \gamma_j \hat{x}_{ij} + \beta_j.$$

With a batch of one and batch statistics, $\mu = x$ and $v = 0$ would give $\hat{x} = 0$: all information lost. In code: `model.train()` versus `model.eval()`.

Theorem 7.11: fusion at inference

At test time BN is the affine map $z \mapsto az + c$ with $a_j = \gamma_j / \sqrt{v_j^{\text{run}} + \epsilon}$, $c_j = \beta_j - a_j \mu_j^{\text{run}}$, so $\text{BN}(W\mathbf{x} + \mathbf{b}) = \text{diag}(\mathbf{a})W\mathbf{x} + (\text{diag}(\mathbf{a})\mathbf{b} + \mathbf{c})$ can be folded into the preceding layer.

Remark 7.12: why BN helps

Reported benefits: easier optimisation, better gradient flow, larger learning rates, less sensitivity to initialisation, mild regularisation through batch noise. The original "internal covariate shift" explanation has been challenged; a smoother loss landscape is an alternative

view. In training mode BN couples samples in the backward pass: with $g_i = \partial L/\partial y_i$ and $r = \sqrt{v+\epsilon}$, $\partial L/\partial x_i = \frac{\gamma}{r}(g_i - \bar{g} - \hat{x}_i\,\overline{g\hat{x}})$.

6 Dropout

Inverted dropout multiplies each unit by $m_i/(1-p)$ with $m_i \sim \text{Bernoulli}(1-p)$ during training (so $\mathbb{E}$ = identity) and does nothing at evaluation. It prevents co-adaptation and acts like training an ensemble of sub-networks (AlexNet applies it to the FC layers).

Theorem 7.13: dropout and activations commute iff positively homogeneous (2021 midterm Q3)

With the same mask, $\phi(H \odot \mathbf{z}) = H \odot \phi(\mathbf{z})$ for all masks iff $\phi(0) = 0$ and $\phi(cz) = c\phi(z)$ for $c = 1/(1-p)$. This holds for ReLU and LeakyReLU, but fails for the sigmoid.

Proof. Coordinatewise, a dropped unit gives $\phi(0)$ versus 0, a kept unit $\phi(cz)$ versus $c\phi(z)$. For $c > 0$, ReLU and LeakyReLU preserve sign cases and scale linearly. For the sigmoid with $p = 0.4$ and $z = 0$: $\sigma(0)/0.6 \approx 0.833 \neq \sigma(0) = 0.5$. ■